

Checked Functional Correctness of an Extracted Jasmin Scalar NTT for HAETAE

Anonymous Submission

Abstract

Number-theoretic transform (NTT) kernels have compact control structure, but their correctness depends on low-level conventions that are easy to state incorrectly: twiddle offsets, inverse scaling, Montgomery signs, and intermediate bounds. We present a replayable EasyCrypt verification of the model extracted from the scalar HAETAE NTT routines written in Jasmin. The proof connects exported procedure models to full 256-point NTT specifications through field, Montgomery, algebraic, and refinement layers, while preserving the implementation’s table padding, inverse final scale, signed Montgomery convention, and coefficient growth. The checked top-level results are two Hoare triples over the extracted forward and inverse wrappers: from a 16-bit signed coefficient representation, the forward wrapper computes the full NTT with a 24-bit output bound, while the inverse wrapper computes the specified inverse transform in Montgomery form with a 16-bit output bound. The result is deliberately kernel-level: it gives checked functional-correctness contracts for the extracted scalar wrappers and makes the boundary to vectorized code, other polynomial routines, full-scheme security, side-channel analysis, round-trip composition, and fresh extraction regeneration explicit.

1 Introduction

Post-quantum signature schemes rely on polynomial arithmetic over finite rings. In HAETAE [8], as in related lattice schemes, this arithmetic is accelerated by a 256-point NTT modulo $q = 64513$. The implementation challenge is that the routine is not merely a textbook transform. It is an in-place, signed, Montgomery-domain computation with a bit-reversed twiddle schedule, final inverse scaling, and machine-word representations. Small mistakes in signs, table offsets, or intermediate bounds can invalidate a proof even when tests pass. A kernel-level proof is therefore useful even without a full signature-scheme proof: a higher-level proof can appeal to this scalar path only after the transform, representation, and bit bounds have been tied to the implementation model under explicit preconditions.

The verification described here targets the scalar Jasmin implementation, with the main routines in `poly.jinc` and exported wrappers in `hpoly.jazz`. Jasmin is designed for high-assurance cryptographic implementations with explicit control over low-level operations [1]. The proof is carried out in EasyCrypt [4] over the extracted model

and culminates in two Hoare statements in `NTTEndToEnd.ec`. The contribution is not a new NTT algorithm, but a checked correspondence between a low-level extracted model and a mathematical specification that uses the same constants and table schedule as the checked extracted routines and tables. The distinction matters for this kernel: the forward implementation skips table index 0, the inverse table reserves index 255 for final scaling, the Montgomery constant is $q^{-1} \bmod 2^{32}$ rather than $-q^{-1}$, and the checked forward bound is 24 bits rather than a smaller informal estimate.

This paper makes three contributions. First, it gives checked end-to-end contracts for the extracted exported Jasmin forward and inverse NTT wrappers; the exact theorem statements appear below and are Hoare triples over the `Hpoly_extract.ec` procedure model. Second, it mechanizes the HAETAE-specific obligations that are usually hidden in informal NTT arguments: the unused forward-table entry, the inverse table’s separate final scale, the $q^{-1} \bmod 2^{32}$ Montgomery convention, and the $16 \rightarrow 24$ forward coefficient-bound growth. Third, it presents a proof organization that keeps algebraic NTT correctness, executable loop correctness, word-array representation, and extracted-code refinement separate until the final Hoare composition.

The contribution is in this integration rather than in a new transform. The checked statement reaches the `Hpoly_extract.ec` model of the public scalar Jasmin entry points, not only an abstract NTT equation. The proof must align the exact implementation schedule with the algebraic transform, because a single off-by-one twiddle index or missing Montgomery factor changes the postcondition. The artifact boundary is therefore part of the claim: the proof establishes functional correctness for the extracted scalar wrappers, while AVX2 code, other polynomial routines, side-channel behavior, full-scheme correctness, and round-trip composition remain outside the displayed top-level theorems.

This division also clarifies what is inherited from existing infrastructure and what is specific to this work. Jasmin and EasyCrypt supply the programming and proof environments. The artifact-specific work is the HAETAE instantiation: the $q = 64513$ field and Montgomery constants, the concrete twiddle-table agreement lemmas, the partial-transform algebra for the full 256-point NTT, and the refinement invariants that preserve byte-array representation and bit bounds through the extracted scalar program.

Detail	Why it matters	Representative evidence
Forward table index 0 is padding	A full-array equality with mathematical <code>zetas</code> is false; only indices 1..255 are consumed	<code>NTT_Fq.ec: jzetas_poly_vals</code>
Inverse index 255 is scale	It is not a butterfly factor, so the invariant and postcondition must isolate final scaling	<code>NTT_Fq.ec: jzetas_inv_255_scaleE</code>
QINV sign convention	The reduction uses q^{-1} , and the REDC postcondition includes R^{-1}	<code>Fq.ec, Montgomery.ec: SREDCp_corr</code>
Unreduced butterfly sums	Coefficients grow during the forward transform, so bit bounds are semantic postconditions	<code>RefJasminNTT.ec, NTTEndToEnd.ec</code>
Byte-array representation	The public routines operate on 1024-byte arrays, not directly on field arrays	<code>NTT_Fq.ec: poly_repr_bound</code>

Table 1: Concrete details that the proof must account for.

The acceptance criterion for the central claim is correspondingly concrete. A reviewer should be able to trace the public Jasmin wrapper names to the extracted program model, the extracted model to the imperative EasyCrypt loops, and those loops to the full-transform specifications, while preserving the stated representation and bit-bound predicates. The paper is organized around that trace rather than around a standalone mathematical derivation of the NTT.

Context and positioning. The work fits into computer-aided cryptography: Jasmin provides a low-level language for high-assurance cryptographic code [1], while EasyCrypt supports mechanized reasoning about cryptographic programs and specifications [4]. Broader work argues for tool-supported links between mathematical claims and executable cryptographic code [3]. This paper contributes a target-specific instance of that methodology for the HAETAE NTT. The semantic foundations are provided by existing tools; the proof value is in closing the HAETAE-specific gap between the extracted module and the EasyCrypt theorems, while preserving constants, consumed table indices, representation predicates, and coefficient bounds from the scalar implementation artifacts. For reviewers, the intended claim is therefore narrow but replayable: the extracted scalar wrappers satisfy the two displayed Hoare contracts under their stated preconditions.

2 Verification Problem

The proof obligation is not simply that the textbook NTT equations are true. The artifact must show that the concrete scalar program model implements the specific full-transform maps used by the EasyCrypt specification, while preserving the representation invariants required by later code. In this setting, a seemingly small mismatch changes the theorem being proved. Table 1 summarizes the implementation details that drive the verification task.

The inputs to the top-level Hoare triples are therefore not arbitrary mathematical polynomials alone. They are byte-array values satisfying `poly_repr_bound`: each 32-bit word decodes to the intended field coefficient and its signed representative stays within the stated bit bound. The outputs are specified in the same representation. This is what lets the proof say both that the mathematical transform is computed and that the result remains in a range compatible with the implementation-level arithmetic model.

The nontriviality is in making these obligations compose. The algebraic proof is most naturally phrased over field arrays and roots of unity; the extracted program operates on byte arrays, signed words, and Montgomery products; and the actual loop schedule consumes only selected table intervals. The proof therefore has to establish more than a high-level NTT identity: it must show that every representation conversion, table lookup, Montgomery multiplication, and coefficient-bound step preserves the exact preconditions needed by the next layer.

3 Implementation Target

The forward routine modeled from `_poly_ntt` is an in-place Cooley–Tukey style loop nest. It initializes `len` to 128 and halves it at each stage. Inside each group it increments `zetasctr` before loading the twiddle, so the forward table entry at index 0 is padding and the implementation consumes `jzetas[1]` through `jzetas[255]`. Each butterfly computes a Montgomery product $t = \text{_fcmul}(zeta, a[j + \text{len}])$, then writes $a[j] + t$ and $a[j] - t$. No modular reduction is inserted between these additions and subtractions; this is why coefficient bounds are part of the functional postcondition.

The inverse routine modeled from `_poly_invntt` runs the inverse loop direction, starting with `len = 1` and doubling it. Its butterfly table is not indexed like the forward table. The implementation reads `jzetas_inv[0..254]` for the 255 inverse butterflies. After the loop it performs a separate final pass using `jzetas_inv[255]`, whose value is the Montgomery-form scale factor -29720 , satisfying the checked identity used to model multiplication by 256^{-1} together with the Montgomery convention.

The field-arithmetic component used for the extracted model comes from `reduce.jinc`, which implements Montgomery reduction [9]. The constant `QINV = 940508161` is $q^{-1} \bmod 2^{32}$, not $-q^{-1}$. The signed Montgomery reduction mirrors the implementation formula

$$(a - tq)/2^{32}, \quad t = \text{int32}(a) \cdot \text{QINV},$$

and the checked EasyCrypt theorem states the Montgomery result, namely multiplication by R^{-1} modulo q for $R = 2^{32}$, together with the signed output bound.

The object verified in EasyCrypt is the extracted procedure model `Hpoly_extract.ec`. The top-level theorems do not reason directly over Jasmin syntax; they reason over

Layer	Main artifacts	Role in the proof
Jasmin code	<code>hpoly.jazz</code> , <code>poly.jinc</code> , <code>reduce.jinc</code> , <code>zetas.jinc</code>	Source of the extracted scalar implementation, tables, and Montgomery arithmetic
Checked extraction	<code>Hpoly_extract.ec</code>	Extracted EasyCrypt model of exported wrappers and cores
Field and words	<code>Fq.ec</code> , <code>GFq.ec</code> , <code>BArray1024.ec</code>	Signed residues, word arrays, field operations
Montgomery	<code>Montgomery.ec</code>	REDC correctness with R^{-1} and signed bounds
Functional NTT	<code>NTT_Fq.ec</code> , <code>NTTFullSpec.ec</code>	Executable loops and mathematical full-transform specs
Algebra	<code>NTTFullAlgebra.ec</code>	Forward/inverse loop algebra and partial-transform invariants
Refinement	<code>RefJasminNTT.ec</code>	Wrapper-to-core equivalence and core-to-loop refinement
End-to-end	<code>NTTEndToEnd.ec</code>	Exported Hoare theorems for <code>poly_ntt_jazz</code> and <code>poly_invntt_jazz</code>

Table 2: Implementation artifacts and proof layers used by the checked result.

exported procedures and internal cores in that extracted model. The refinement layer then connects those procedures to the executable EasyCrypt NTT loops. This distinction is important for the claim: source-level names identify the implementation target, while the checked Hoare triples are statements about the extracted model and its imported semantics.

4 Specification and Algebra

The EasyCrypt development separates executable loop models from mathematical full transform specifications. The specifications operate on length-256 arrays over $\mathbb{Z}_q$, with $q = 64513$, and use the checked root $\zeta = \mathbf{zroot} = 426$. In the specification, $\zeta^{256} = -1$, and the forward and inverse maps in `NTTFullSpec.ec` are defined by

$$\mathbf{full_ntt}(p)[i] = \sum_{j=0}^{255} p[j] \zeta^{(2 \text{br}(i)+1)j}$$

and

$$\mathbf{full_invntt}(p)[i] = 256^{-1} \sum_{j=0}^{255} p[j] \zeta^{-(2 \text{br}(j)+1)i},$$

where `br` is 8-bit reversal. The implementation-facing procedures in `NTT_Fq.ec` then give executable loop models over field coefficients, plus the representation predicate `poly_repr_bound`. This predicate connects the byte-array representation used by extracted Jasmin code to a polynomial over $\mathbb{Z}_q$, while also recording a signed bit bound for each coefficient.

The bit reversal in the formulas is therefore part of the checked full-transform specification, not only an artifact of loop order.

The forward twiddle array is defined mathematically as

$$\mathbf{zetas}[k] = \zeta^{\text{bsrev}_8(k)} \quad (0 \leq k < 256),$$

where `bsrev8` is 8-bit reversal. At stage s , group g uses table index $k = 2^s + g$, matching the implementation’s increment-before-use counter. The checked table lemma for `jzetas` is intentionally restricted to indices $1 \leq i < 256$: index 0 is the implementation padding value 0, while the mathematical Montgomery image of `zetas[0]` is R . Thus the table agreement claim is a Montgomery-form, consumed-index statement rather than a raw equality between complete arrays.

The algebraic layer in `NTTFullAlgebra.ec` proves that the executable loops implement the full specifications in `NTTFullSpec.ec`. A key technical point is that the proof does not rely on a loose informal “butterfly applies a root” argument. Instead it uses segment-indexed partial-transform definitions and split lemmas for the low and high halves of each butterfly segment. This matches the nested loop structure of the implementation: the invariant knows the current length, segment start, and twiddle position, and states exactly which portion of the transform has been established.

The inverse proof follows the same discipline but with the inverse loop order and the final scale. The scale is not another butterfly twiddle: it is isolated as index 255 of `jzetas_inv`. The executable inverse procedure is therefore specified as the inverse NTT followed by the Montgomery-form scaling represented in EasyCrypt by `array256_mont`.

5 Mechanized Proof Strategy

The mechanized development is organized so that each layer changes exactly one view of the program. This is important for auditability: a reviewer can inspect whether the algebra, the imperative loops, or the low-level representation is responsible for a given claim. The procedure names in the following chain are the checked `Hpoly_extract.ec` models of the exported Jasmin code and its internal core. The forward path has the following shape:

```

poly_ntt_jazz
↓ poly_ntt_jazz_ref
poly_ntt
↓ poly_ntt_core_ref
NTT_ntt
↓ ntt_full_ntt
full_ntt

```

The inverse path is analogous, using the corresponding inverse wrapper, core-refinement, and algebra lemmas. The final file `NTTEndToEnd.ec` composes these facts into the public Hoare triples by consequence.

Each arrow in this chain is a checked lemma with a narrower interface than the whole theorem. The wrapper lemmas remove the public-call shell, the core refinement lemmas connect extracted code to executable EasyCrypt

loops, and the algebra lemmas connect those loops to the full-transform specifications. This decomposition prevents the end-to-end theorem from hiding either a representation conversion or an algebraic NTT step inside an informal equivalence.

The algebraic step is where the transform-specific proof work resides. The specification file defines the forward and inverse maps as array-valued sums over $\mathbb{Z}_q$. The algebra file proves that the imperative loops compute those sums using stage invariants over partial transforms. For the forward transform, the key split lemmas decompose a length- 2^{k+1} partial sum into the two length- 2^k halves combined by a butterfly. For the inverse transform, the proof follows the reverse loop order and accounts for the final multiplication by the scale entry. This structure is why the proof can talk about the implementation's nested loops without rewriting the top-level theorem in terms of machine words.

The refinement step is deliberately separate. It relates the extracted Jasmin procedures to the coefficient-array loop model while tracking representation and bit bounds. The wrapper lemmas are simple, because the public wrappers inline to the core routines. The core refinement lemmas are larger: they maintain `poly_repr_bound`, prove that each loaded twiddle is the intended field element on the actually consumed index range, and use the signed Montgomery theorem to justify each call to the field multiplication routine.

6 End-to-End Theorems

The final EasyCrypt file `NTTEndToEnd.ec` composes the algebraic loop correctness theorems with refinement from the extracted procedure model. The two exported results are the following Hoare triples, quoted in the same shape as the checked file. Informally, the first theorem says that the extracted exported forward procedure implements the mathematical full NTT and tracks the necessary coefficient growth; the second says that the extracted exported inverse procedure implements the mathematical inverse NTT in Montgomery representation and restores a 16-bit bound.

```
lemma haetae_jasmin_ntt_correct p :
  hoare [Hpoly_extract.M.poly_ntt_jazz :
    NTT_Fq.poly_repr_bound rp p 16 ==>
    NTT_Fq.poly_repr_bound res
    (NTTFullSpec.full_ntt p) 24].
```

```
lemma haetae_jasmin_invntt_correct p :
  hoare [Hpoly_extract.M.poly_invntt_jazz :
    NTT_Fq.poly_repr_bound rp p 16 ==>
    NTT_Fq.poly_repr_bound res
    (NTT_Fq.array256_mont
     (NTTFullSpec.full_invntt p)) 16].
```

In these Hoare triples, `rp` is the byte-array input to the extracted procedure model and `res` is the returned byte array. The predicate `poly_repr_bound a p b` asserts both representation agreement with the abstract polynomial p and a signed b -bit bound on the decoded coefficients. Thus the postconditions combine a functional statement about the computed transform with a range statement about the

representation left for later code. The inverse theorem deliberately postconditions `array256_mont (full_invntt p)` rather than `full_invntt p`: the verified routine returns the Montgomery-form result specified by the checked model.

The bit indices in these predicates have their ordinary signed-word meaning in the development: `bw32 a sz` is the interval $-2^{sz} \leq \text{to_sint}(a) < 2^{sz}$. Thus the forward theorem's $16 \rightarrow 24$ statement says that unreduced butterfly arithmetic is allowed to grow but remains within the checked 24-bit signed range. The inverse theorem's $16 \rightarrow 16$ statement says that the inverse loop and final Montgomery-form scale return to the checked 16-bit signed range for the Montgomery representation in the postcondition.

The forward theorem says that, for any abstract polynomial p , if the input byte array `rp` represents p with signed coefficients bounded by 16 bits, then the extracted exported routine returns a byte array representing `full_ntt(p)` with a 24-bit bound. The inverse theorem has a different but equally important shape: from a 16-bit represented input, it returns the Montgomery representation of `full_invntt(p)`, again bounded by 16 bits.

These bounds are part of the theorem, not post-hoc measurements. The forward proof tracks growth through the unreduced additions and subtractions of the butterfly network and ends at 24 bits. The inverse proof combines its loop invariant with the final Montgomery scale to recover a 16-bit bound. This distinction matters: a false forward claim such as $16 \rightarrow 23$ or an inverse claim that ignores Montgomery scaling would not match the checked statements.

7 Refinement Argument

The refinement layer proves that the extracted Jasmin procedures behave like the EasyCrypt loop models. In `RefJasminNTT.ec`, the wrapper lemmas `poly_ntt_jazz_ref` and `poly_invntt_jazz_ref` show that the exported wrappers call the internal extracted cores. The larger core lemmas `poly_ntt_core_ref` and `poly_invntt_core_ref` relate those cores to the executable EasyCrypt NTT procedures. These proofs must account for the representation boundary: Jasmin uses 32-bit words and 1024-byte arrays, while the functional specification uses 256 field coefficients. Lemmas in `NTT_Fq.ec` bridge these views through `barray256_to_poly`, `word_to_coeff`, and `poly_repr_bound`.

Montgomery multiplication is another boundary. The low-level code multiplies signed 32-bit values in 64-bit registers and applies signed Montgomery reduction. The EasyCrypt model clones a signed-reduction theory with $q = 64513$, $R = 2^{32}$, $q^{-1} = 940508161$, and $R^{-1} = 50386$. Its core correctness lemma proves that signed REDC returns a value congruent to aR^{-1} modulo q and within the signed range used by the rest of the proof. This is the reason the NTT proof can reason in $\mathbb{Z}_q$ while the implementation

Claim checked	Boundary crossed	Representative evidence
Exported forward wrapper satisfies final contract	Extracted wrapper to spec	NTTEndToEnd.ec: haetae_jasmin_ntt_correct
Exported inverse wrapper satisfies final contract	Extracted wrapper to spec	NTTEndToEnd.ec: haetae_jasmin_invntt_correct
Forward loop computes <code>full_ntt</code>	Executable loop to math spec	NTTFullAlgebra.ec: ntt_full_ntt
Inverse loop computes <code>full_invntt</code>	Inverse loop plus final scale	NTTFullAlgebra.ec: invntt_full_invntt
Forward wrapper equals core	Exported model to extracted core	RefJasminNTT.ec: poly_ntt_jazz_ref
Inverse wrapper equals core	Exported model to extracted core	RefJasminNTT.ec: poly_invntt_jazz_ref
Forward core refines loop model	Extracted core to EasyCrypt proc	RefJasminNTT.ec: poly_ntt_core_ref
Inverse core refines loop model	Extracted core to EasyCrypt proc	RefJasminNTT.ec: poly_invntt_core_ref
Consumed twiddle entries agree with specs	Concrete tables to field arrays	NTT\Fq.ec: jzetas_poly_vals, jzetas_inv_255_scaleE
Signed Montgomery REDC is correct	Machine arithmetic to field arithmetic	Montgomery.ec: SREDcP_corr

Table 3: Examples of checked obligations used by the end-to-end result.

remains close to the C and Jasmin arithmetic.

8 Proof Obligations and Evidence

The proof effort is concentrated around three obligations that correspond to common failure modes in low-level NTT arguments. The first is representation agreement. A coefficient of the mathematical polynomial is an element of $\mathbb{Z}_q$, but the implementation stores a signed 32-bit representative in a 1024-byte array. The predicate `poly_repr_bound` bundles these two facts: each word decodes to the corresponding field coefficient and its signed integer representative is below a stated bit bound. The top-level theorems therefore state both the transformed polynomial and the output size.

The second obligation is table agreement. The forward table cannot be stated as a full-array equality between `jzetas` and the Montgomery image of `zetas`, because index 0 differs by design. The checked lemmas therefore prove the equality only on the consumed interval $1 \leq i < 256$. The inverse table has a different shape: indices 0 through 254 are butterfly factors, and index 255 is reserved for the final scale. Treating this final entry as another butterfly factor would give the wrong program invariant.

The third obligation is arithmetic agreement. The Montgomery-reduction model is signed: it sign-extends 32-bit inputs, multiplies in 64-bit registers, and uses an arithmetic right shift. The EasyCrypt proof connects that behavior to field multiplication by proving the signed REDC theorem with the R^{-1} factor. Consequently the field-level proof uses the same Montgomery convention as the implementation, and the range conclusion supplies the fact needed by the NTT bound invariants.

The reviewer-facing evidence is the theorem dependency chain ending in `NTTEndToEnd.ec`: the end-to-end file im-

ports the algebra and refinement layers, composes their lemmas by consequence, and exposes the two Hoare triples quoted above. The final file makes this composition explicit through small intermediate lemmas: for each direction, one lemma combines the core-refinement theorem with the algebraic loop theorem, and a second lemma moves from the internal core to the exported wrapper. The displayed final theorems are consequences of these checked compositions, not independent restatements of the specification.

The top-level claim is replayed by invoking `easycrypt compile` on `NTTEndToEnd.ec` with the development and extraction libraries on the load path. Successful compilation checks the imported dependency chain under the configured EasyCrypt environment, including the extracted module used by the refinement proof. This replay is evidence for the displayed Hoare triples, not a performance evaluation and not an extraction-regeneration check. It establishes that the field, Montgomery, table-agreement, algebraic loop, and refinement layers compose to the two displayed contracts for all inputs satisfying the 16-bit `poly_repr_bound` precondition. A mismatch between the prose and the named Hoare triples, table lemmas, or Montgomery lemma would be an error in the paper, not an alternative interpretation of the verified result.

9 Discussion

The development separates the mathematical transform, the executable loop, and the low-level program until the final composition step. This organization keeps changes in one layer from rewriting the obligations of the others. The mathematical specification describes `full_ntt` and `full_invntt`; the executable EasyCrypt loops expose the implementation’s control structure; and the refinement layer handles word-level details without reopening the algebra.

This separation also makes several excluded claims visible. The checked development does not state that the entire forward table equals the Montgomery image of the mathematical `zetas` array, because that statement is false at index 0. It does not state that Montgomery reduction preserves the input residue without an R^{-1} factor, because that would be the wrong convention. It does not state that the forward transform maps 16-bit inputs to 23-bit outputs, because the checked postcondition is 24 bits. Each excluded statement changes the shape of a proof obligation and would break the connection to the checked artifact.

Compared with test-based validation of the NTT, the proof covers the full preconditioned input space of the displayed Hoare triples. This matters for cryptographic arithmetic because boundary cases arise near signed representative limits and table offsets. The proof is still not a replacement for all implementation validation: it assumes the semantics provided by the Jasmin extraction and EasyCrypt libraries, and it targets the scalar extraction used by the checked development. Constant-time behavior, compiler back-end issues, and integration with

the full signature scheme are separate questions.

The proof architecture suggests a practical pattern for other lattice kernels with similar structure. First, fix the arithmetic convention, including the Montgomery factor, before proving loop properties. Second, prove table agreement only on the indices consumed by the implementation. Third, express loop progress with partial-transform invariants that follow the concrete loop nest. Finally, compose extracted-code refinement only after the algebraic loop theorem is available. The HAETAE proof is one instance of this pattern: it shows that the low-level conventions are not side conditions to be checked informally, but named obligations needed by the final Hoare contracts.

The scope is deliberately kernel-level. The result complements, rather than replaces, full protocol or security proofs: it discharges a concrete implementation-level arithmetic obligation for a performance-critical primitive. This is the level at which table offsets, Montgomery conventions, and bit-growth arguments are most likely to diverge from a high-level mathematical description.

10 Related Work

NTTs are finite-field analogues of FFT-style evaluation and interpolation methods [10]. They are now a standard implementation technique for lattice-based cryptography, including schemes such as CRYSTALS-Kyber [5], CRYSTALS-Dilithium [6], and HAETAE [8]. This paper does not contribute a faster transform or a new parameter set. Its subject is the narrower question of how to connect one concrete scalar NTT implementation to a checked functional specification without losing the implementation details that determine correctness.

Verified cryptographic implementation projects have shown several ways to connect executable arithmetic to formal specifications. HACL* [11] verifies a broad C cryptographic library in the F*/Low* ecosystem, while Fiat-Crypto [7] generates efficient finite-field arithmetic from Coq-checked specifications. The present work is smaller in scope but different in shape: it starts from hand-written low-level Jasmin routines, uses the extracted EasyCrypt model, and proves that the resulting scalar NTT wrappers satisfy HAETAE-specific transform contracts. It is therefore neither a general verified library nor a code generator; it is a focused proof that a concrete lattice-arithmetic kernel matches its specification despite low-level representation, table-indexing, and Montgomery-domain details.

EasyCrypt [4] and the broader computer-aided cryptography literature [3] provide the setting for mechanized reasoning about cryptographic programs and claims. The present development is not a game-based security proof for the HAETAE signature scheme. It sits one layer below such arguments: a functional-correctness proof for the concrete arithmetic implementation that security proofs and algorithm descriptions would otherwise have to trust informally. The proof therefore complements, rather than competes with, protocol-level verification.

For NTTs and polynomial arithmetic, the closest comparison is the Jasmin/EasyCrypt verification of Kyber/ML-KEM implementations [2]. That work is broader than the result here because it connects Jasmin implementations for a KEM to EasyCrypt specifications. The present paper is narrower and should be read as a kernel proof, not as a full-scheme verification. Its technical delta is the HAETAE-specific instantiation: modulus 64513, Montgomery constant $q^{-1} \bmod 2^{32}$, a full 256-point transform, a forward table with unused padding at index 0, and an inverse table whose index 255 is final scaling rather than a butterfly twiddle. These details are small enough to look like engineering choices, but they determine the shape of the checked lemmas and final Hoare postconditions. This is the main difference from a paper that only states a textbook NTT proof or reports tests against reference vectors.

11 Boundary and Trust

The checked development establishes two functional-correctness contracts: one for the scalar Jasmin forward NTT wrapper and one for the scalar inverse wrapper. Both contracts are stated over the EasyCrypt model of the exported procedures and the representation predicate used throughout the proof. This is the intended theorem boundary: the result is a statement about extracted scalar procedure models satisfying the two displayed Hoare triples.

Several adjacent claims are intentionally outside that boundary. The result does not cover an AVX2 implementation, the full HAETAE signature scheme, side-channel security, or other exported polynomial routines such as `poly_basemul_jazz`. It also does not expose a standalone checked round-trip theorem stating that the verified forward routine followed by the verified inverse routine is the identity on all represented inputs. Such a statement would need to combine the two displayed Hoare triples with a checked algebraic identity relating `full_invntt` and `full_ntt`, plus the Montgomery-output convention of the inverse theorem. The current paper does not depend on that missing composition theorem: the forward and inverse contracts are stated and replayed separately.

The trusted base is correspondingly explicit. The proof relies on the semantics of EasyCrypt, the Jasmin extraction used to obtain the EasyCrypt model of the source procedures, the standard fixed-size word and array libraries imported by the development, and the adequacy of the functional specifications `full_ntt` and `full_invntt` as the intended HAETAE NTT contracts. Regenerating `Hpoly_extract.ec` from the Jasmin sources and comparing it with the extraction used by the proof is an artifact-integrity check, not part of the theorem replay reported in this paper. Validation of a particular assembler, compiler backend, linker, or calling-context integration is a separate engineering task.

12 Conclusion

This verification connects an extracted scalar polynomial-arithmetical model to a precise formal specification without abstracting away the details that usually cause NTT proof errors: signed Montgomery reduction, table padding, inverse-table indexing, final scaling, and coefficient growth. The replayed top-level EasyCrypt file composes table agreement, Montgomery correctness, algebraic loop correctness, and refinement into two concrete Hoare contracts for the extracted models of the exported HAETAE NTT wrappers. The main lesson is methodological as well as artifact-specific: for lattice-arithmetical kernels, the proof must follow the implementation’s representation, table schedule, and bounds closely enough that small low-level conventions remain visible in the final theorem.

References

- [1] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Gregoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub. Jasmin: High-assurance and high-speed cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1807–1823. ACM, 2017.
- [2] J. B. Almeida, M. Barbosa, G. Barthe, B. Gregoire, V. Laporte, J.-C. Lechenet, T. Oliveira, H. Pacheco, M. Quaresma, P. Schwabe, A. Sere, and P.-Y. Strub. Formally verifying kyber: Episode IV: Implementation correctness. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):164–193, 2023.
- [3] M. Barbosa, G. Barthe, K. Bhargavan, B. Blanchet, C. Cremers, K. Liao, and B. Parno. Sok: Computer-aided cryptography. In *42nd IEEE Symposium on Security and Privacy*, pages 777–795. IEEE, 2021.
- [4] G. Barthe, F. Dupressoir, B. Gregoire, C. Kunz, B. Schmidt, and P.-Y. Strub. EasyCrypt: A tutorial. In *Foundations of Security Analysis and Design VII*, volume 8604 of *LNCS*, pages 146–166. Springer, 2013.
- [5] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle. CRYSTALS-Kyber: A cca-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy*, pages 353–367. IEEE, 2018.
- [6] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehle. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, 2018.
- [7] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala. Simple high-level code for cryptographic arithmetic: With proofs, without compromises. In *2019 IEEE Symposium on Security and Privacy*, pages 1202–1219. IEEE, 2019.
- [8] HAETAE Team. HAETAE: Lattice-based digital signature algorithm. Submission to the Korean Post-Quantum Cryptography Competition, 2022. <https://kpmc.or.kr/competition.html>.
- [9] P. L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.
- [10] J. M. Pollard. The fast Fourier transform in a finite field. *Mathematics of Computation*, 25(114):365–374, 1971.
- [11] J. K. Zinzindohoue, K. Bhargavan, J. Protzenko, and B. Beurdouche. HACl*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806. ACM, 2017.